

Създаване на приложения с използване на сложни данни. Работа с речници и множества.

I. Теоретична постановка

1. Речник

Речникът е набор от обекти, достъпът до които се осъществява не чрез индекс, а чрез ключ. Речниците могат да съдържат данни от различен тип и притежават неограничена степен на влагане. Елементите в речника са разположени в произволен порядък. За достъп до тях трябва да се използва ключ. Операциите конкатенация, сечение и повторение тук не се поддържат. Операциите с речник включват достъп до елемент по ключ, промяна на елемент по ключ, проверка за наличие на даден ключ (in), определяне броя на ключовете, добавяне или изтриване на елемент.

Създаване на речник: `dict = {`
 `"toys": 16,`
 `"books": 30,`
 `"pens": 15 }`

- Достъп до елемент по ключ

```
>>> dict["toys"]  
>>> 16
```

- Промяна на елемент по ключ

```
>>> dict["pens"] = 20  
>>> dict  
{'toys': 16, 'books': 30, 'pens': 20}
```

- Проверка за наличие на даден ключ

```
>>> "books" in dict  
>>> True
```

- Определяне броя на ключовете

```
>>> len(dict)  
>>> 3
```

- Добавяне на елемент по ключ

```
>>> dict["balls"] = 5  
>>> dict  
{'toys': 16, 'books': 30, 'pens': 20, 'balls': 5}
```

- Изтриване на елемент по ключ

Прави се с помощта на оператор `del`.

```
>>> del dict["books"]  
>>> dict  
{'toys': 16, 'pens': 20, 'balls': 5}
```

- Обхождане на елементи на речник

Два много полезни метода за работа с речник са `keys()` и `values()`. Първият връща обект `dict_keys`, който съдържа всички ключове, записани в речника, а вторият връща обект `dict_values` с всички стойности от речника.

```
>>> keys=dict.keys()
```

```
>>> keys
dict_keys(['toys', 'pens', 'balls'])
>>> vals=dict.values()
>>> vals
dict_values([16, 20, 5])
```

Обхождането на речника може да се организира с метод `keys()` и цикъл `for`:

```
>>> for key in dict.keys():
    print(key, " ", dict[key])
```

```
toys 16
pens 20
balls 5
```

- Сортиране на речник

Речникът е неподредена структура от данни. Затова при обхождане на речник неговите ключове се извеждат в произволен порядък. Той може да се сортира по ключове. За целта първо трябва да се изведат всички ключове от речника в списък и след това този списък да се сортира.

```
>>> keys = list(dict.keys())
>>> keys.sort()
>>> for key in keys:
    print(key, " ", dict[key])
```

```
balls 5
pens 20
toys 16
```

Пример 1: Да се напише програма, която въвежда 5 низа за ключ и 5 цели числа за стойност от клавиатурата и ги записва в речник. Програмата да извърши следните обработки: търси избран ключ в речника, променя стойността на елемент от речника по неговия ключ, изтрива елемент от речника по ключ, извежда всички ключове от речника, извежда всички стойности от речника, сортира речника по ключове.

```
dict={} # Създава празен речник
for i in range(0,5):
    key=input("Ключ: ")
    val=int(input("Стойност: "))
    dict[key]=val # Запълва речника с елементи
print(dict)
x=input("Въведете търсен ключ: ")
if x in dict: # Търси елемент в речника
    print(f"{x} е в речника")
else:
    print(f"{x} не е в речника")
y=input("Въведете ключ на елемент за промята: ")
z=int(input("Въведете нова стойност: "))
if y in dict:
```

```
dict[y]=z          # Променя елемент на речника по ключ
print(dict)
else:
    print(f"{y} не е в речника")
y=input("Въведете ключ на елемент за изтриване: ")
if y in dict:
    del dict[y]      # Изтрива елемент на речник по ключ
    print(dict)
else:
    print(f"Индекс {y} не е в речника")
print("Ключовете в речника са: ")
keys=dict.keys()    # Извлича ключовете от речника
print(keys)
print("Стойностите в речника са: ")
vals=dict.values()  # Извлича стойностите от речника
print(vals)
print("Сортиран речник: ")
lst=list(dict.keys())
lst.sort()          # Сортира списъка с ключовете от речника
for key in lst:
    print(key, " ", dict[key])
```

2. Множество

Множеството е неподредена съвкупност от уникални елементи, т.е. в него не може да има повтарящи се елементи. Множествата поддържат операциите получаване на дължина, обединение, разлика, пресичане на множества и проверка за наличие.

- Създаване на множество: `s1 = set ([1, 2, 3, 4, 5])`
- Получаване на дължина

```
>>> len(s1)
>>> 5
```

- Обединение на множества – с оператор `|`

```
>>> s1 = set ([1, 2, 3, 4, 5])
>>> s2 = set ([5, 6, 7])
>>> s3 = s1 | s2
>>> s3
{1, 2, 3, 4, 5, 6, 7}
```

При обединение на множества в новото множество попадат само уникалните елементи. Няма повторения!

- Разлика на множества – с оператор `-`

```
>>> s1 = set ([1, 2, 3, 4, 5])
>>> s2 = set ([3, 4, 5])
>>> s3 = s1 - s2
>>> s3
{1, 2}
```

Премахва от множеството *s1* елементите, които съществуват и в множеството *s1*, и в множеството *s2*.

- Пресичане (сечение) на множества – с оператор *&*

```
>>> s1 = set([1, 2, 3])
```

```
>>> s2 = set([3, 4, 5])
```

```
>>> s3 = s1 & s2
```

```
>>> s3
```

```
{3}
```

Резултатът са елементите, които се съдържат и в двете множества.

- Операторът *^* връща елементите на двете множества, изключвайки еднаквите елементи в тях

```
>>> s1 = set([1, 2, 3])
```

```
>>> s2 = set([3, 4, 5])
```

```
>>> s3 = s1 ^ s2
```

```
>>> s3
```

```
{1, 2, 4, 5}
```

- Проверка за наличие се прави с оператор *in*

```
>>> s1 = set([1, 2, 3])
```

```
>>> 3 in s1
```

```
True
```

```
>>> 4 in s1
```

```
False
```

- Полезни методи на множества са

add(<Елемент>) Добавя <Елемент> в множеството

remove(<Елемент>) Премахва (<Елемент>) от множеството. Дава грешка ако (<Елемент>) не съществува в множеството!

clear() Изчиства множеството

Пример 2: Да се напише програма, която създава две множества от 5 цели числа, въведени от клавиатурата и извършва следните операции с тях: обединение на двете множества, разлика на двете множества, пресичане на двете множества, премахване на избран елемент от първо множество и изчистване на двете множества.

```
s1=set()                              # Създаване на празно множество
```

```
s2=set()                              # Създаване на празно множество
```

```
print(type(s1))
```

```
print(type(s2))
```

```
for i in range(0,5):
```

```
    s=int(input("Елемент I множество: "))
```

```
    s1.add(s)                          # Добавяне на елемент
```

```
print("I множество: ")
```

```
print(s1)
```

```
for i in range(0,5):
```

```
    s=int(input("Елемент II множество: "))
```

```
    s2.add(s)                          # Добавяне на елемент
```

```
print("II множество: ")
print(s2)
s3=s1/s2
print("Обединение на множества I и II: ")
print(s3)
s3=s1-s2
print("Разлика на множества I и II: ")
print(s3)
s3=s1&s2
print("Пресичане на множества I и II: ")
print(s3)
s=int(input("Елемент, който да се премахне от I множество: "))
if s in s1:
    s1.remove(s)      # Премахване на елемент от първо множество
else:
    print(f"{s} не се съдържа в множество I")
print(s1)
print("Изчистване на множество I:")
s1.clear()           # Изчистване на множество
print(s1)
print(s2)
print("Изчистване на множество II:")
s2.clear()           # Изчистване на множество
print(s2)
```

II. Задачи за самостоятелна работа

1. Направете промени в кода на Пример 1, така че извеждането на съдържанието на речника да се прави с обхождане на елементите му по ключ, а не с функция print.
2. Направете промени в кода на Пример 2, така че да се определя размера на двете създадени множества.
3. Напишете програма на Python, която създава речник от 7 въведени от потребителя низа и съответстващите им цели числа и извършва следните обработки: разпечатва съдържането на речника, определя на кой ключ съответства най-голямото въведено цяло число от записаните в речника, заменя стойността въведена за избран ключ с друга въведена от потребителя, изтрива стойност от речника по нейния ключ и добавя нова стойност в речника заедно с нейния ключ.